

Azure Learning Plan - 2 Weeks Full Plan

Overview

Lộ trình 2 tuần full coverage từ Azure Fundamentals đến demo cuối cùng. Mỗi ngày gồm: đọc lý thuyết → hands-on lab → ghi chú.

Tech stack công ty: Web, API, Lambda, RDS, ECS **AWS** → **Azure mapping** được cover đầy đủ

AWS → Azure Quick Reference

AWS	Azure
Lambda	Azure Functions
ECS / EKS	Azure Container Apps / AKS
EC2	Azure Virtual Machines
RDS (PostgreSQL, MySQL, SQL)	Azure Database for PostgreSQL / MySQL / SQL
S3	Azure Blob Storage
API Gateway	Azure API Management
CloudWatch	Azure Monitor / Application Insights
IAM	Azure Entra ID (Azure AD)
SQS	Azure Service Bus Queues
SNS	Azure Service Bus Topics / Event Grid
Secrets Manager	Azure Key Vault
ECR	Azure Container Registry
Route 53	Azure DNS
CloudFront	Azure Front Door

Week 1: Fundamentals + Core Services

Day 1: Azure Fundamentals

Goal: Navigate được Azure Portal, hiểu core concepts

Lý thuyết (2-3h):

- ☐ Azure Portal overview
- ☐ Subscriptions, Resource Groups, Regions
- ☐ Azure CLI installation & basic commands
- ☐ Azure Resource Manager (ARM) concepts
- ☐ Azure AD (Entra ID) introduction
- ☐ RBAC roles (Owner, Contributor, Reader, User Access Admin)

Hands-on (2h):

```
# Azure CLI basics
az login
az account list
az account set --subscription "subscription-id"
az group create --name myResourceGroup --location southeastasia
az resource list --resource-group myResourceGroup
```

Task: Tạo Resource Group, thử deploy ARM template đơn giản

Resource: [AZ-900 Learning Path](#)

Day 2: App Service - Web Hosting

Goal: Deploy và chạy được web app

Lý thuyết (2h):

- ☐ App Service Plans (Free, Shared, Basic, Standard, Premium, Isolated)
- ☐ Web App vs API App vs Mobile App
- ☐ Deployment methods (GitHub Actions, Azure DevOps, FTP, ZIP)
- ☐ Deployment slots (staging, production)
- ☐ Application settings vs Connection strings
- ☐ Custom domains và SSL certificates
- ☐ VNet integration (private access)
- ☐ Scale up vs Scale out (autoscaling)

Hands-on (3h):

```
# Tạo App Service Plan và Web App
az appservice plan create --name myAppServicePlan --resource-group myResourceGroup --sku S1 --location southeastasia
az webapp create --name myUniqueWebApp --resource-group myResourceGroup --plan myAppServicePlan --runtime "NODE|18 LTS"
az webapp deployment source config --name myUniqueWebApp --resource-group myResourceGroup --repo-url "https://github.com/your/repo"
```

Task: Deploy một REST API (Node.js/Python/.NET) lên App Service với staging slot

Auto-scaling (App Service):

```
# Enable autoscale
az monitor autoscale create --name myAutoscale --resource-group myResourceGroup --resource myAppServicePlan --resource-type Microsoft.Web.AppServices

# Add scale rule (HTTP queue depth)
az monitor autoscale rule create --resource-group myResourceGroup --autoscale-name myAutoscale --condition "HttpQueueDepth > 100" --

# Add scale rule (CPU)
az monitor autoscale rule create --resource-group myResourceGroup --autoscale-name myAutoscale --condition "CpuPercentage > 70" --sc

# Set min/max instances
az monitor autoscale update --name myAutoscale --resource-group myResourceGroup --min-count 2 --max-count 10 --count 2
```

Resource: [Microsoft Learn: Deploy a web application](#)

Day 3: Azure Functions - Serverless

Goal: Viết và deploy serverless functions

Lý thuyết (2h):

- ☐ Consumption vs Premium vs Dedicated plans (pricing model)
- ☐ Triggers: HTTP, Timer (CRON), Blob, Queue, Event Hub, Cosmos DB
- ☐ Input/Output bindings
- ☐ Durable Functions (orchestration patterns)
- ☐ Application Insights cho monitoring
- ☐ Local development (VS Code, Azure Functions Core Tools)
- ☐ Dependency injection (nếu .NET)

Hands-on (3h):

```
# Install Azure Functions Core Tools
npm install -g azure-functions-core-tools@4

# Create project
func init myFunctionApp --python
cd myFunctionApp
func new --name HttpTriggerExample --template "HTTP trigger"

# Run locally
func start

# Deploy
az functionapp create --name myFunctionApp --resource-group myResourceGroup --runtime python --runtime-version 3.11 --functions-versions latest
az functionapp deployment source config --name myFunctionApp --resource-group myResourceGroup --repo-url "your-repo" --branch main
```

Task:

1. HTTP-triggered function (GET/POST)
2. Timer-triggered function (chạy mỗi 5 phút)
3. Blob-triggered function (process image upload)

Resource: [Microsoft Learn: Build serverless applications](#)

Day 4: Containers - Azure Container Apps

Goal: Deploy containerized app (ECS equivalent)

Lý thuyết (2h):

- ☐ Container Apps vs App Service containers vs ACI vs AKS
- ☐ Container Apps revision management
- ☐ Ingress configuration (public vs private)
- ☐ Scale rules (HTTP concurrency, CPU/memory, custom)
- ☐ Health probes (liveness, readiness)
- ☐ Azure Container Registry (ACR) basics
- ☐ Docker basics (images, containers, Dockerfile)

Hands-on (3h):

```
# Login to ACR
az acr login --name myAcrRegistry

# Build and push image
docker build -t myAcrRegistry.azurecr.io/myapp:latest .
docker push myAcrRegistry.azurecr.io/myapp:latest

# Create Container Apps environment
az containerapp env create --name myEnv --resource-group myResourceGroup --location southeastasia

# Create Container App
az containerapp create --name myContainerApp --resource-group myResourceGroup --environment myEnv --image myAcrRegistry.azurecr.io/myapp:latest

# Scale
az containerapp update --name myContainerApp --resource-group myResourceGroup --min-replicas 1 --max-replicas 10
```

Task: Containerize một simple API và deploy lên Container Apps với auto-scaling

Resource: [Microsoft Learn: Deploy to Azure Container Apps](#)

Day 5: Databases - Azure SQL & PostgreSQL

Goal: Sử dụng được managed databases (RDS equivalent)

Lý thuyết (2h):

- ☐ Azure SQL Database vs SQL Managed Instance vs SQL on VM

- ☐ Single database vs Elastic pool
- ☐ DTU vs vCore purchasing model
- ☐ Azure Database for PostgreSQL - Flexible Server
- ☐ Azure Database for MySQL - Flexible Server
- ☐ Connection pooling (Azure SQL, pgBouncer for PostgreSQL)
- ☐ Firewall rules (server-level vs database-level)
- ☐ Azure AD authentication
- ☐ Backup strategy (point-in-time restore, geo-restore, geo-replication)
- ☐ Read replicas

Hands-on (3h):

```
# Azure SQL Database
az sql server create --name mySqlServer --resource-group myResourceGroup --location southeastasia --admin-user sqladmin --admin-pass
az sql db create --name myDatabase --resource-group myResourceGroup --server mySqlServer --service-objective S0

# Configure firewall
az sql server firewall-rule create --resource-group myResourceGroup --server mySqlServer --name AllowAzureIPs --start-ip-address 0.0

# Connect string
az sql db show-connection-string --name myDatabase --server mySqlServer --client adonet

# Azure Database for PostgreSQL
az postgres flexible-server create --name myPostgresServer --resource-group myResourceGroup --location southeastasia --admin-user po
```

Task: Kết nối web app với Azure SQL Database, run migration

Auto-scaling (Azure SQL Database):

```
# Enable autocaling (vCore model)
az sql db update --name myDatabase --resource-group myResourceGroup --server mySqlServer \
  --auto-pause-delay 60 \
  --min-capacity 10 \
  --max-capacity 50 \
  --capacity 20

# Or using elastic pool
az sql elastic-pool create --name myElasticPool --resource-group myResourceGroup --server mySqlServer \
  --sku-name Standard_Pool \
  --min-per-database-capacity 0 \
  --max-per-database-capacity 50 \
  --per-database-min-capacity 10 \
  --per-database-max-capacity 50

# Azure Database for PostgreSQL auto-scaling
az postgres flexible-server update --name myPostgresServer --resource-group myResourceGroup \
  --storage-size 256 \
  --sku-name Standard_D4s_v3 \
  --high-availability Enabled
```

Resource: [Microsoft Learn: Work with relational data in Azure](#)

Week 2: Integration + DevOps + Demo

Day 6: API Management

Goal: Front-end API với APIM (API Gateway equivalent)

Lý thuyết (2h):

- ☐ API Management tiers (Consumption, Developer, Basic, Standard, Premium)
- ☐ APIM components: Gateway, Developer Portal, Azure Portal
- ☐ Import API (OpenAPI/Swagger, WSDL, WADL)
- ☐ API Operations
- ☐ Products, Groups, Subscriptions

- ☐ Policies: rate limit, cache, transform, cors, authentication
- ☐ Named values (like environment variables)
- ☐ API versioning (URL path, header, query string)
- ☐ API revision (for safe updates)

Hands-on (3h):

```
# Create APIM
az apim create --name myApimService --resource-group myResourceGroup --location southeastasia --sku-name Consumption

# Import OpenAPI
az apim api import --resource-group myResourceGroup --service-name myApimService --api-id my-api --path "users" --value "https://mya

# Add rate limit policy
az apim api policy set --resource-group myResourceGroup --service-name myApimService --api-id my-api --value '{
  "rate-limit": {
    "calls": "100",
    "renewal-period": "60"
  }
}'
```

Task:

1. Import existing API lên APIM
2. Add rate limiting (100 req/min)
3. Enable CORS
4. Setup API key authentication

Resource: [Microsoft Learn: Implement API Management](#)

Day 7: Blob Storage + Azure Key Vault

Goal: File storage (S3 equivalent) và secrets management

Lý thuyết (2h):

Blob Storage:

- ☐ Storage Account (general-purpose v2)
- ☐ Containers vs Blobs
- ☐ Access tiers (Hot, Cool, Archive)
- ☐ Blob types (Block, Page, Append)
- ☐ SAS tokens và Shared Access Policies
- ☐ Lifecycle management policies
- ☐ Azure Storage Explorer

Key Vault:

- ☐ Key Vault tiers (Standard vs Premium)
- ☐ Secrets vs Keys vs Certificates
- ☐ Access policies vs RBAC
- ☐ Managed identities (system-assigned, user-assigned)
- ☐ Soft delete, purge protection

Hands-on (3h):

```
# Storage Account
az storage account create --name mystorageaccount --resource-group myResourceGroup --location southeastasia --sku Standard_LRS
az storage container create --name mycontainer --account-name mystorageaccount

# Upload/download blob
az storage blob upload --container-name mycontainer --name myfile.txt --file myfile.txt --account-name mystorageaccount
az storage blob download --container-name mycontainer --name myfile.txt --file downloaded.txt --account-name mystorageaccount

# Generate SAS token
az storage blob generate-sas --container-name mycontainer --name myfile.txt --account-name mystorageaccount --permissions r --expiry

# Key Vault
az keyvault create --name myKeyVault --resource-group myResourceGroup --location southeastasia
az keyvault secret set --vault-name myKeyVault --name "DatabasePassword" --value "MySecretPassword123"
az keyvault secret show --vault-name myKeyVault --name "DatabasePassword"
```

Task:

1. Upload images to Blob Storage, generate SAS URL
2. Store connection string in Key Vault
3. Access Key Vault secret from App Service using managed identity

Resource:

- [Microsoft Learn: Store and access data in Azure](#)
- [Microsoft Learn: Secure cloud data](#)

Day 8: Virtual Network (VNet)

Goal: Network isolation và private access cho Azure resources

Lý thuyết (2h):

- ☐ Virtual Network (VNet) concepts
- ☐ Subnets (public vs private)
- ☐ Network Security Groups (NSG)
- ☐ VNet peering
- ☐ Private Endpoints
- ☐ VNet integration (App Service, Functions)
- ☐ Private DNS zones
- ☐ VPN Gateway / ExpressRoute basics

Hands-on (3h):

```
# Create VNet
az network vnet create --name myVnet --resource-group myResourceGroup --location southeastasia --address-prefixes 10.0.0.0/16

# Create subnets
az network vnet subnet create --name mySubnet --vnet-name myVnet --resource-group myResourceGroup --address-prefixes 10.0.1.0/24
az network vnet subnet create --name myPrivateSubnet --vnet-name myVnet --resource-group myResourceGroup --address-prefixes 10.0.2.0

# Create NSG
az network nsg create --name myNsg --resource-group myResourceGroup --location southeastasia
az network nsg rule create --name myNsgRule --nsg-name myNsg --resource-group myResourceGroup --priority 100 --direction Inbound --a

# Associate NSG with subnet
az network vnet subnet update --name mySubnet --vnet-name myVnet --network-security-group myNsg --resource-group myResourceGroup

# Create Private Endpoint for App Service
az network private-endpoint create --name myPrivateEndpoint --resource-group myResourceGroup --vnet-name myVnet --subnet myPrivateSu

# Enable VNet integration on App Service
az webapp config set --name myWebApp --resource-group myResourceGroup --vnet-name myVnet --subnet mySubnet
```

Task:

1. Create VNet với 2 subnets (public, private)

2. Create NSG và associate với subnet
3. Enable VNet integration trên App Service
4. Test private access via Private Endpoint

Resource: [Microsoft Learn: Azure Virtual Network](#)

Day 9: DevOps - CI/CD + Monitoring

Goal: Automate deployment và monitor application

Lý thuyết (2h):

CI/CD:

- ☐ GitHub Actions basics (workflow, jobs, steps, actions)
- ☐ Azure login action
- ☐ Deploy to App Service action
- ☐ Deploy to Functions action
- ☐ Deploy to Container Apps action
- ☐ Azure Pipelines basics (nếu dùng Azure DevOps)
- ☐ Environment variables, secrets in CI/CD

Monitoring:

- ☐ Azure Monitor (metrics, logs)
- ☐ Application Insights
- ☐ Log Analytics workspaces
- ☐ Alerts (metric, log, availability)
- ☐ Application Insights SDK integration
- ☐ Distributed tracing

Hands-on (3h):

```
# GitHub Actions workflow example for App Service
# .github/workflows/deploy.yml
name: Deploy to Azure App Service

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: azure/login@v1
        with:
          creds: ${ secrets.AZURE_CREDENTIALS }
      - uses: azure/webapps-deploy@v3
        with:
          app-name: myWebApp
          slot-name: production
          package: .
```

```
# Enable Application Insights
az monitor app-insights component create --app myAppInsights --location southeastasia --resource-group myResourceGroup

# Create alert
az monitor metrics alert create --name "HighCPUAlert" --resource-group myResourceGroup --condition "cpu > 80" --description "CPU usage high"
```

Task:

1. Setup GitHub Actions deploy to App Service
2. Add Application Insights to app
3. Create alert for failed requests

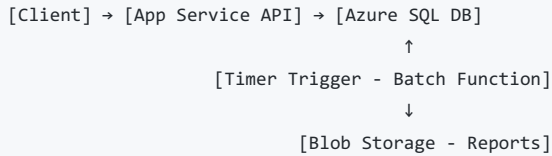
Resource:

- [Microsoft Learn: Build and deploy to Azure](#)

Day 10: Final Demo Project

Demo: Simple CRUD + Batch Processing

Architecture



Components

1. App Service - CRUD API (1 tiếng)

- REST API: GET/POST/PUT/DELETE cho /tasks
- Kết nối Azure SQL Database
- Task model: { id, title, description, status, createdAt }

2. Azure Function - Batch Processor (1 tiếng)

- Timer trigger (chạy mỗi 5 phút)
- Query: `SELECT * FROM tasks WHERE status = 'pending'`
- Generate report (VD: count by status)
- Save report JSON to Blob Storage

Deployment Steps

```
# Setup
az group create --name demoResourceGroup --location southeastasia

# App Service
az appservice plan create --name demoPlan --resource-group demoResourceGroup --sku S1 --location southeastasia
az webapp create --name demoCrudApi --resource-group demoResourceGroup --plan demoPlan --runtime "NODE|18 LTS"

# SQL Database
az sql server create --name demoSqlServer --resource-group demoResourceGroup --location southeastasia --admin-user demouser --admin-password 'P@ssw0rd!'
az sql db create --name demoDb --resource-group demoResourceGroup --server demoSqlServer --service-objective S0

# Storage
az storage account create --name demostorageacct --resource-group demoResourceGroup --location southeastasia --sku Standard_LRS
az storage container create --name reports --account-name demostorageacct

# Function
az functionapp create --name demoBatchProcessor --resource-group demoResourceGroup --runtime node --runtime-version 18 --functions-v2
```

Demo Flow

1. CRUD API - Tạo/đọc/sửa/xóa tasks


```
# Create task
curl -X POST https://demoCrudApi.azurewebsites.net/api/tasks \
-H "Content-Type: application/json" \
-d '{"title": "Task 1", "description": "Description", "status": "pending"}'

# Get all tasks
curl https://demoCrudApi.azurewebsites.net/api/tasks

# Update task
curl -X PUT https://demoCrudApi.azurewebsites.net/api/tasks/1 \
-H "Content-Type: application/json" \
-d '{"title": "Task 1 Updated", "status": "completed"}'

# Delete task
curl -X DELETE https://demoCrudApi.azurewebsites.net/api/tasks/1
```

2. **Batch Function** - Mỗi 5 phút:
- Query all tasks
 - Generate status summary report
 - Save to Blob: `reports/daily-summary-2026-05-04.json`
3. **Verify**:
- Check API: `GET /api/tasks`
 - Check Blob: report file created

Success Criteria

- ☐ CRUD API works (create, read, update, delete)
- ☐ Batch Function runs on timer
- ☐ Report saved to Blob Storage
- ☐ Logs visible in Application Insights

Weekly Summary

Week 1 Summary

Day	Topic	AWS Equivalent
1	Azure Fundamentals	IAM, AWS Organizations
2	App Service	Elastic Beanstalk, EC2
3	Azure Functions	Lambda
4	Azure Container Apps	ECS/EKS
5	Azure SQL / PostgreSQL	RDS

Week 2 Summary

Day	Topic	AWS Equivalent
6	API Management	API Gateway
7	Blob Storage + Key Vault	S3 + Secrets Manager
8	Virtual Network	VPC, Security Groups
9	CI/CD + Monitoring	CodePipeline + CloudWatch

Resources

1. [Microsoft Learn](#)
 2. [Azure Portal](#)
 3. [Azure Architecture Center](#)
 4. [Azure SDKs](#)
 5. [Azure CLI Reference](#)
 6. [AZ-204 Exam Prep](#)
-

Next Steps (Sau 2 tuần)

- CI/CD full automation
- Infrastructure as Code (ARM templates / Terraform / Bicep)
- Security hardening (Defender for Cloud, network security)
- Cost optimization (reserved instances, auto-shutdown)
- AZ-204 certification
- Multi-region deployment